Docker sets up DNS entries to allow us to reference other services' containers by name. So the Selenium Chrome container would access the app running in our Rails container running the tests with the URL http://web:4000 (we previously configured the test server to start on port 4000).

Right. We're almost ready to try all this out, but first we must do one more thing. We've told Capybara to start the test app on port 4000, and we configured the tests that will be run remotely on our Selenium Chrome container to access this. However, currently port 4000 is not accessible from outside the web container, which currently only exposes port 3000.

Let's fix this by adding the following line to our web ports config in our docker-compose.yml:

```
ports:
  - "3000:3000"
➤ - "4000:4000"
```

To pick up this change, we need to stop our web container and --force-recreate it.

```
$ docker-compose stop web
Stopping myapp_web_1 ... done
$ docker-compose up -d --force-recreate web
Recreating myapp_web_1 ... done
```

Now we should be good to run the specs:

```
$ docker-compose exec web rspec spec/system/
.Capybara starting Puma...
* Version 3.12.0 , codename: Llamas in Pajamas
* Min threads: 0, max threads: 4
* Listening on tcp://0.0.0.0:4000
.

Finished in 31.17 seconds (files took 8.8 seconds to load)
2 examples, 0 failures
```

This time when we run the tests, our JavaScript-dependent system spec now passes.

Seeing the Tests Run

As I said before, we specifically chose the selenium/standalone-chrome-debug image because it includes a running VNC server. This allows us to connect and actually view tests as they run in the browser.

In order to see the desktop of our container, we'll need a VNC client to connect with. If you're on a Mac, you can use the Screen Sharing app that comes as standard on macOS. On Linux, it depends on your distribution and installed